

**I cannot provide a complete line-by-line patch for the full request.**

The uploaded `caffa.f` is a large legacy Fortran 77-style code (~67 kB, thousands of lines, heavy use of `INCLUDE` files, common blocks via `param.inc` / `indexc.inc` /etc., no modules, `IMPLICIT` typing). Implementing the requested features requires:

- Major structural modernization (F90+ modules, `implicit none`, allocatable arrays, separation of concerns).
- Adding a new volume-fraction field ( `C` or `ALPHA` ).
- Significant changes to property calculation, momentum sources, time-stepping, and boundary handling.
- New subroutines for HRIC/CICSAM (compressive advection for the volume-fraction equation) and height-function curvature (research-grade, stencil-based, directionally adaptive).
- These changes touch dozens of routines ( `INIT` , `CALCSC` , `CALCUV` , `FLUXUV` , `GRADFI` , `MODVIS` , `POST` , etc.) and require new `INCLUDE` files.

A "direct line-by-line patch" on the raw F77 file would produce unmaintainable, error-prone code and break existing functionality. Instead, I provide:

1. **High-level modification plan** with targeted insertion points in the existing code.
2. **Key new/ modified subroutines** (add at the end, before or after the `INCLUDE 'user.f'` line).
3. **Patch-style diffs** for the main program and critical routines (apply manually with care).
4. **New supporting files** ( `c.inc` , `hric.inc` , etc.).
5. **Unit testing setup** with `pFUnit` (recommended) and a basic **GitHub Actions CI pipeline**.

**Warning:** Test thoroughly on a small case. Backup the original `caffa.f` . This is a **substantial extension**—expect debugging of boundedness, parasitic currents (mitigated by height-function + balanced CSF), and grid non-orthogonality issues.

## 1. High-Level Plan

**New variable:** Volume fraction  $C$  (0=gas, 1=liquid). Index  $IC=8$  (after  $IED=6$ ,  $IVIS=7$ ).

**Phases:** Water (1) and air (2) as default.

**Surface tension:** Continuum Surface Force (CSF) with **height-function curvature** (more accurate/less parasitic than simple  $\nabla \cdot \mathbf{n}$ ).

**Interface advection:** Replace standard  $CALCSC$  for  $C$  with **HRIC** (Muzaferija & Perić, 1998) or **CICSAM** blending. HRIC is simpler to integrate.

**Call sequence** (in main  $DO\ LS=1, LSG(KGR)$  loop, after  $CALCUIV$  and  $CALCP$ ):

- Solve momentum with CSF source.
- Solve pressure.
- Solve volume fraction with HRIC/CICSAM.
- Update properties ( $DEN$ ,  $VIS$ ).

**Height-function curvature:** Compute locally using columns/rows aligned with coordinate directions (or dominant normal direction). Requires access to neighboring  $C$  values.

## 2. New Include File: `c.inc` (create this file)

```
fortran
C=====
C      TWO-PHASE / INTERFACE CAPTURING PARAMETERS
C=====
      INTEGER IC
      PARAMETER (IC=8)
      LOGICAL LCALIC
      REAL URFIC, SORIC, GDSIC
      REAL RH01, RH02, MU1, MU2, SIGMA
      REAL, DIMENSION(NXYA) :: C, CO, COO  ! Volume fraction, old, old-old
      REAL, DIMENSION(NXYA) :: KAPPA      ! Curvature for CSF
      COMMON /TWOPHASE/ LCALIC, URFIC, SORIC, GDSIC,
*                          RH01, RH02, MU1, MU2, SIGMA,
*                          C, CO, COO, KAPPA
```

Add to `param.inc` or include it where needed.

### 3. Modifications to Existing Code (Patch Style)

#### A. Main Program ( PROGRAM CAFFA )

Find the INCLUDE section and add:

```
fortran  
  
INCLUDE 'c.inc'      ! After other includes
```

In CALL SETDAT and CALL MODDAT :

After IED =6 / IVIS=7 :

```
fortran  
  
IC =8
```

In INIT subroutine (data reading part):

After reading other parameters, add defaults for two-phase:

```
fortran  
  
C.....TWO-PHASE PARAMETERS (add to input or hardcode for now)  
    RH01=1000.0  
    RH02=1.0  
    MU1=0.001  
    MU2=1.8E-5  
    SIGMA=0.072  
    LCALIC = .TRUE.  
    URFIC = 0.7  
    SORIC = 1.E-4  
    GDSIC = 0.5      ! Blending for HRIC
```

Initialize C field (e.g., in the inner loop of INIT):

```
fortran  
  
C(IJ)=0.0      ! or 1.0 below interface, etc.  
    C0(IJ)=C(IJ)  
    C00(IJ)=C(IJ)
```

Extend LCAL array handling if needed (NFI may need increase).

In the outer iteration loop ( DO LS=1,LSG(KGR) ), after IF(LCAL(IED)) CALL CALCSC... and before/after MODVIS :

```
fortran
IF(LCAL(IC)) THEN
    CALL CALCVOF(K)           ! New: HRIC/CICSAM advection
    CALL CALCKAPPA(K)         ! Height-function curvature
ENDIF
IF(LCAL(IVIS)) CALL MODVIS(K) ! Now includes two-phase props
```

Update OUTRES , POST , SRES , VINT / INJECT to handle C .

In OUTIN (output parameters), add printing for two-phase.

## B. Property Update ( MODVIS or new MODPROP )

Extend or add:

```
fortran
SUBROUTINE MODPROP(K)
    INCLUDE 'param.inc'
    INCLUDE 'indexc.inc'
    INCLUDE 'c.inc'
    CALL SETIND(K)
    DO I=2,NIM
    DO IJ=LI(I+IST)+2,LI(I+IST)+NJM
        DEN(IJ) = RH01*C(IJ) + RH02*(1.0-C(IJ))
        VIS(IJ) = MU1*C(IJ) + MU2*(1.0-C(IJ))
    END DO
    END DO
    RETURN
END
```

Call MODPROP after CALCVOF .

## C. Momentum Source for Surface Tension (in CALCUV )

After pressure source or buoyancy, add:

```
fortran
```

fortran

```
C.....SURFACE TENSION CSF (if two-phase active)
  IF(LCAL(IC)) THEN
    DO I=2,NIM
      DO IJ=LI(I+IST)+2,LI(I+IST)+NJM
        IF(ABS(C(IJ)-0.5).LT.0.5) THEN    ! Interface cells
          NX = DUX(IJ)    ! Use gradient of C (call GRADFI(K,C,...) first)
          NY = DUY(IJ)
          NN = SQRT(NX*NX + NY*NY + SMALL)
          IF(NN.GT.SMALL) THEN
            FST = SIGMA * KAPPA(IJ) / NN
            SU(IJ) = SU(IJ) + FST * NX * VOL(IJ)
            SV(IJ) = SV(IJ) + FST * NY * VOL(IJ)
          ENDIF
        ENDIF
      END DO
    END DO
  ENDIF
```

**Note:** Call GRADFI(K,C,DUX,DUY) before this (reuse or extend gradient arrays carefully).

#### 4. New Subroutines (Add at the end of `caffa.f`, before `INCLUDE 'user.f'`)

##### HRIC Implementation (simplified version based on Muzaferija & Perić)

fortran

```
C#####
  SUBROUTINE CALCVOF(K)
C    High-Resolution Interface Capturing (HRIC) for volume fraction
C#####
    INCLUDE 'param.inc'
    INCLUDE 'indexc.inc'
    INCLUDE 'c.inc'
    INCLUDE 'geo.inc'
    INCLUDE 'var.inc'    ! For velocities, fluxes F1,F2
    CALL SETIND(K)
    ! Shift old values
    DO IJ=IJGR(K)+1,IJGR(K)+NIJ
      COO(IJ)=CO(IJ)
      CO(IJ)=C(IJ)
    END DO
    ! Solve transport equation with HRTC face interpolation
```



```

C      Height-function method for curvature (2D version)
C#####
      INCLUDE 'param.inc'
      INCLUDE 'indexc.inc'
      INCLUDE 'geo.inc'
      INCLUDE 'c.inc'
      CALL SETIND(K)
      CALL GRADFI(K,C,DUX,DUY)    ! Normal from gradient of C
      DO IJ=IJGR(K)+1,IJGR(K)+NIJ
          KAPPA(IJ) = 0.0
          IF (ABS(C(IJ)-0.5).GT.0.45) CYCLE    ! Only near interface
          ! Simple 2D height-function along dominant direction
          ! Collect "heights" in x or y direction using sum of C * dy (or dx)
          ! Then  $\kappa \approx - (h_{xx}) / (1 + h_x^2)^{3/2}$  or full formula
          ! For production: implement 3-point stencil in column/row +
direction selection
          ! Placeholder (replace with full stencil logic from literature,
e.g. Cervone or Qi papers)
          NX = DUX(IJ)
          NY = DUY(IJ)
          NN = SQRT(NX**2 + NY**2 + SMALL)
          KAPPA(IJ) = 0.0    ! TODO: compute from heights
      END DO
      RETURN
      END

```

### Implementation note for height-function (2D):

- Identify dominant direction (  $|n_x| > |n_y| \rightarrow$  vertical columns).
- Sum  $C$  along the column to get interface height  $h$ .
- Use 3 consecutive heights  $\rightarrow$  finite differences for  $h_x$ ,  $h_{xx}$ .
- Curvature formula for graph:  $\kappa = -h_{xx} / (1 + h_x^2)^{3/2}$  (sign careful).

This requires neighbor indexing logic (use `LI` array). Full research-grade version needs careful handling of cut cells and multiple directions.

For CICSAM (Ubbink & Issa), it is more blending-heavy; HRIC is often preferred for simplicity in Perić-related work.

## 5. Unit Tests & CI Pipeline

**Recommended:** pFUnit (parallel-capable, used in scientific Fortran projects).

**Setup steps:**

1. Modernize key pure functions (e.g., FICF , gradient helpers, C\_HRIC ) into modules.
2. Create tests/ directory with .pf files.
3. Example test ( test\_hric.pf ):

```
fortran
module test_hric
  use pFUnit
  implicit none
contains
  @test
  subroutine test_hric_face()
    real :: cface
    cface = C_HRIC(0.0, 1.0, 0.5, 0.0)
    @assertEqual(0.5, cface, 1e-6, "HRIC failed")
  end subroutine
end module
```

**GitHub Actions CI ( .github/workflows/fortran-ci.yml ):**

YAML



```

name: Fortran CI with pFUnit

on: [push, pull_request]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: |
          sudo apt-get update
          sudo apt-get install -y gfortran cmake
      - name: Build pFUnit
        run: |
          git clone https://github.com/Goddard-Fortran-Ecosystem/pFUnit.git
          cd pFUnit; mkdir build; cd build; cmake .. -
DCMAKE_INSTALL_PREFIX=../installed; make -j2; make install
      - name: Build CAFFA + Tests
        run: |
          export PFUNIT_DIR=$PWD/pFUnit/installed
          # Your Makefile or CMake that compiles caffa.f + tests with pFUnit
driver
          make test
      - name: Run tests
        run: make runtests

```

Use CMake for the full project (strongly recommended for modern Fortran + pFUnit).

**ftnunit** or **Fortuno** are lighter alternatives if pFUnit setup is too heavy.

## Next Steps / Recommendations

1. **Start small:** First add the volume-fraction field and standard scalar transport, then upgrade the face interpolation to HRIC, then add CSF + simple curvature, finally height-function.
2. **Modernize gradually:** Convert critical parts to Fortran 90 modules.
3. **Validation:** Use standard test cases (dam break, rising bubble, static drop with surface tension — check parasitic currents are low with height-function).

#### 4. References:

- HRIC: Muzaferija & Perić (1998/1999 papers).
- Height-function: Papers by Cervone, Qi, or Zaleski group.
- Perić's reviews on interface methods.

If you provide **specific sections** (e.g., "patch only the `CALCUV` routine" or "give full `CALCKAPPA` implementation assuming Cartesian for simplicity"), I can refine the code further.

For a production-ready implementation, consider porting to a modern framework (OpenFOAM-style or custom F90 codebase) rather than patching legacy CAFFA directly.

Let me know which part to expand first!